

Machine Learning Project Report

Date: 07/02/2025

University: Alpen-Adria-Universität Klagenfurt

Course: Machine Learning (700.851, 24W)

Group Members & Responsibilities:

Vladyslav Uhrik - Data selection & preparation

Andrii Zhukov, Dmytro Velychko - Model selection & training

Egor Pichugin, Tykhin Biriukov - front-end, back-end

Topic: Develop a CNN for Image Classification using Transfer Learning

Difficulty: Average

Dataset: Augmented Alzheimer MRI Dataset V2

Models: EfficientNet B4, ResNet152, DenseNet-169

[GitHub Link](#)

Table of Contents:

1. Objectives
2. Data Selection
3. Data Preparation
4. Model Selection
5. Model Re-Training
6. Logging and Plotting Results
7. Results
8. Stack

1. Objectives

Select and prepare a dataset, select a model, re-train and fine-tune to achieve maximum performance while efficiently handling limited computational resources.

2. Data Selection

The reason we chose to do Image Classification as our topic is how visual the project is. For example, by printing out a grid of random pictures from the dataset in Jupyter we could already see many features of the dataset: picture quality, various imperfections and inconsistencies which could directly influence the accuracy of future predictions of our model. In this way we went through a couple of datasets before settling on the Alzheimer brain MRI's.

Dataset Name: Augmented Alzheimer MRI Dataset V2

Source: [Kaggle](#)

Size: 400 MB

Train pictures: 23788

Test pictures: 10192

Total pictures: 33980

Classes: 4

Resolution: 380x380

3. Data Preparation

Following that, we made a custom transformation to correct the deviations in color and contrast.

Custom transformation:

Resizing: 380x380 Center crop

Normalization: Custom calculation for a dataset using a python script

Other transformations: Grayscale

4. Model Selection

Initial Choice: EfficientNetB4 ~19B params

Pros: Efficient scaling and computation

Cons: Slow training, high memory requirements

Max. Accuracy: 70.49%

Second Choice: ResNet151 ~60B params

Pros: Widely used, recommended by the teacher

Cons: Heavy computation, can overfit

Max. Accuracy: 99.31%

Final Choice: DenseNet-169 ~14B params

Pros: Parameter-efficient, highest accuracy achieved, Most balanced in our case

Max. Accuracy: 99.73%

5. Model Re-training

Optimizer: Adam

Learning rate: 0.001

Loss Function: Cross Entropy Loss

Scheduler: ReduceLROnPlateau

Factor: 0.5

Patience: 2

Threshold: 0.01

Batch size: 32

Number of epochs: 25

Hardware:

1) T4 GPU (Google Colab)

2) RTX 3050 Laptop GPU (locally, vscode)

3) RTX 4090 (Vast.ai - teacher's recommendation)

6. Logging and Plotting Results

Metrics: train loss, train accuracy, test loss, test accuracy, F1-score, precision, recall, confusion matrix, top mistakes plot

Libraries Used: Matplotlib, torchmetrics

7. Results

Final Performance:

Train loss: 0.00094

Test loss: 0.00332

Train acc: 99.97%

Test acc: 99.91%

8. Stack

Fronted:

- Vue.js
- Naive-UI

Backend:

- Node.js
- Python (script)

Pipeline:

Once the user uploads an X-ray of the brain through the user interface, it is sent to the backend, which triggers a Python script to process the image using a trained model.